

ENSEA

Beyond Engineering

REAL-TIME OPERATING SYSTEMS

Session 1 – Introduction to FreeRTOS

Laurent Fiack – laurent.fiack@ensea.fr

Bureau D212

Today

- Last course: a superloop, a BSP, an app, events, state machines – everything on one big `while(1)`
- Today: replace the loop by an **operating system**, and look at what it gives us
- The code you will write barely changes. What changes is *who decides what runs when*

ENSEA
Beyond Engineering

Today

1. What Is an Operating System?
2. What's Real-Time ?
3. Managing Time
4. Our First Tasks
5. Context Switch
6. Tasks
7. FreeRTOS and CMSIS-OS
8. FreeRTOS API Basics

ENSEA
Beyond Engineering

What an OS is for

- Let **one processor** run *several programs* at once
  - A *program* is called a **task**: an infinite loop, independent from `main`
  - Also called *thread* or *process*, depending on the OS
- **Manage time**: the OS grants *timeslots* to tasks
  - Tasks run *one after the other*, never truly simultaneously (one core)
- **Manage memory**: dynamic allocation
- **Synchronise and connect tasks**: semaphores, mutexes, queues
- Some OSes also provide drivers, file systems, networking – but not all of them

ENSEA
Beyond Engineering

Kernel vs. OS

- **Kernel**: the core of the OS – scheduling, synchronisation, memory
- **OS**: kernel + a software collection (drivers, file system, network stack, shell...)

In a RTOS

Most of the time you only get the **kernel**. Drivers, protocols and the rest are yours to bring (here: the HAL and our BSP).

ENSEA
Beyond Engineering

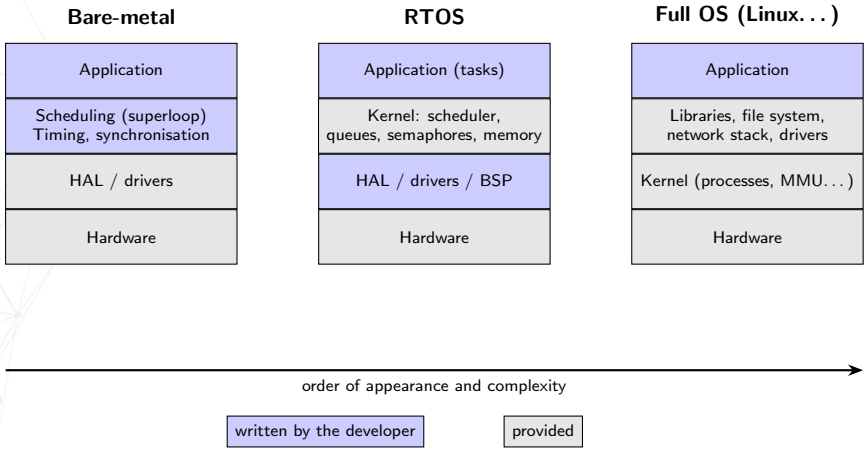
A family of operating systems

	Examples	**Typical target**
Single-task	HAL, MS-DOS	small MCU, simple devices
Multi-task, desktop/server	Linux, Windows, macOS	PC, server
Multi-task, embedded (IoT)	tinyOS, Contiki	connected objects
Multi-task, **real-time**	**FreeRTOS**, RTEMS, VxWorks	MCU

- Two axes: single vs. multi-task, and "general purpose" vs. embedded vs. **real-time**

ENSEA
Beyond Engineering

Software evolution: bare-metal → RTOS → OS



- Going right, the developer writes less infrastructure – and gains less control

ENSEA
Beyond Engineering

Real-time vocabulary

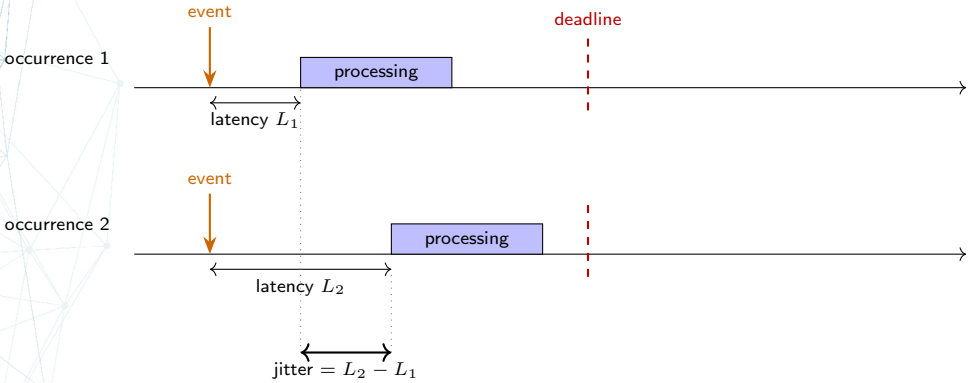
- **Deadline**: the instant before which the response must be produced
- **Latency**: delay between an event and the start of its processing
- **Jitter**: variation of the latency from one occurrence to the next
- **WCET** (Worst Case Execution Time): longest time a piece of code can take

Real-time $\neq$ fast

Real-time means **predictable**: the deadlines are met, every time.

ENSEA
Beyond Engineering

Latency, deadline, jitter



- Both responses meet the deadline, but the latency is not the same – the question is: *by how much, at worst?*

Real-time constraints

- **Hard** real-time: missing a deadline is a failure (airbag, motor control)
- **Soft** real-time: missing a deadline degrades quality (audio glitch, laggy UI)
- Hard real time means a **predictable** and **reproducible** response time
  - Jitter (=latency dispersion) of a few percent

Warning

A real-time OS does **not** guarantee that *your application* is real-time. A task that takes too long, or a bad priority choice, misses deadlines on any RTOS.

Demo: the same project, two tasks

```c
static app_t app;
static bsp_t bsp;

void task_bsp(void *unused) { for (;;) { bsp_process(&bsp); } }
void task_app(void *unused) { for (;;) { app_process(&app); } }

int main(void) {
    /* HAL_Init(), SystemClock_Config(), MX_..._Init() */
    bsp_init(&bsp, &app.events);
    app_init(&app, &bsp);

    xTaskCreate(task_bsp, "BSP", 256, NULL, 1, NULL);
    xTaskCreate(task_app, "APP", 256, NULL, 1, NULL);

    vTaskStartScheduler();      /* never returns */
}
```

- Live demo first. We unpack every line of it during this session.

ENSEA
Beyond Engineering

The scheduler and the tick

- The **scheduler** picks which task runs next
- The **tick**: a periodic interrupt (`SysTick` on Cortex-M) that wakes the scheduler
  - `configTICK_RATE_HZ` = 1000 Hz here: the scheduler gets control every 1 ms
  - Fast tick: more reactive, but more time spent in the kernel
- The time unit of FreeRTOS is the **tick**
  - `vTaskDelay(10)` waits 10 ticks – not 10 ms!
  - Use `pdMS_TO_TICKS(10)` to convert from milliseconds

ENSEA
Beyond Engineering

Three scheduling strategies

Cooperative

- A task runs until *it* gives control back.
  - That is our superloop: `bsp_process` then `app_process`, each one returns
  - One slow function blocks everyone

Time-sliced (round-robin)

- The scheduler cuts time into slices (1 tick) and rotates between tasks of the *same priority*.

Preemptive

- As soon as a higher-priority task becomes ready, it takes the CPU *immediately*.

- Let's compare them on the same scenario

ENSEA
Beyond Engineering

Scenario

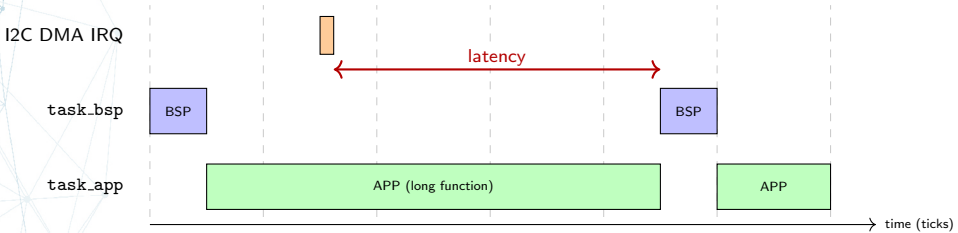
- `task_bsp`: reads inputs, forwards events – must react **quickly**
- `task_app`: menu, game logic, screen drawing – long computations
- An I^2C DMA transfer to the display completes: its interrupt (IRQ) fires and the BSP has work to do

Question

How long does it take before `task_bsp` handles it, in each strategy?

ENSEA
Beyond Engineering

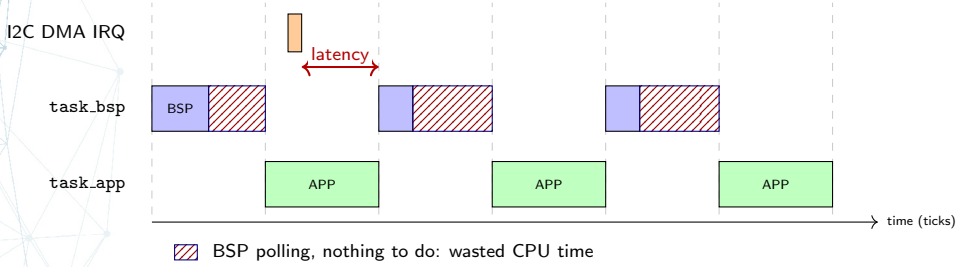
Cooperative



- The ISR only sets a flag. The BSP sees it when `app_process` finally *returns*
- Latency $\approx$ the longest function of the other tasks

ENSEA
Beyond Engineering

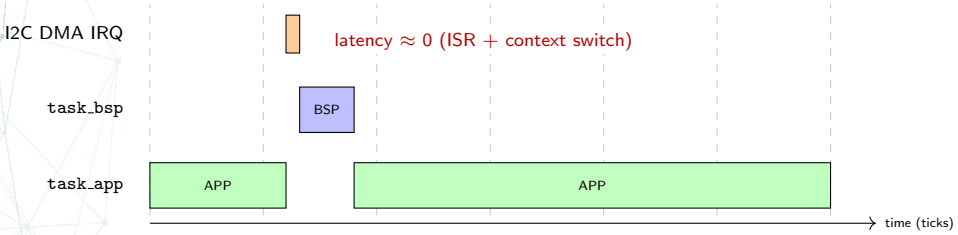
Time-sliced



- Latency is bounded by the slice – better than cooperative, and nobody can hog the CPU
- But the CPU is split **blindly**: the BSP wastes its slice polling, and it is as (un)important as the APP
- With N tasks, the worst case grows to $N - 1$ slices

ENSEA
Beyond Engineering

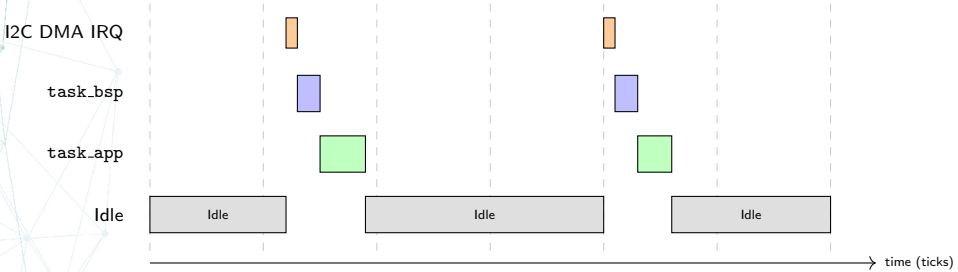
Preemptive



- The ISR wakes the BSP; being **higher priority**, it runs right after the ISR, interrupting the APP
- Latency is short and independent of what the APP is doing – this is the real-time strategy
- It needs tasks that can **block** (wait for an event) instead of polling: next sessions

ENSEA
Beyond Engineering

Preemptive, in real life



- A real system spends most of its time **waiting**: tasks are Blocked, the Idle task runs
- An event wakes the BSP, which wakes the APP; both do a short job, then block again
- Latency stays tiny *and* the CPU is free the rest of the time – room for more tasks, or for sleeping

ENSEA
Beyond Engineering

Before / after

Superloop

```c
int main(void) {
    /* HAL + MX init */
    bsp_init(&bsp, &app.events);
    app_init(&app, &bsp);

    while (1) {
        bsp_process(&bsp);
        app_process(&app);
    }
}
```

RTOS

```c
void task_bsp(void *unused) {
    for (;;) bsp_process(&bsp);
}
void task_app(void *unused) {
    for (;;) app_process(&app);
}
int main(void) {
    /* HAL + MX init, bsp_init, app_init */
    xTaskCreate(task_bsp, ...);
    xTaskCreate(task_app, ...);
    vTaskStartScheduler();
}
```

The architecture pays off

bsp/ and app/ do not change. Only `main.c` does.

ENSEA
Beyond Engineering

Initialisation stays in `main`

- `bsp_init` and `app_init` run **before** `vTaskStartScheduler()`
- Otherwise we lose control of the order: if `task_app` started first, the BSP would not be initialised yet
- Tasks only contain the *processing*, never the setup

ENSEA
Beyond Engineering

Why app and bsp are `static`

- `vTaskStartScheduler()` resets the **MSP** (main stack pointer) to the top of the stack (`prvPortStartFirstTask` in `port.c`)
- Tasks run on the **PSP** (process stack pointer), each on its own stack allocated in the FreeRTOS heap
- From then on the MSP is only used by interrupts, which overwrite the local variables of `main`

Consequence

Anything shared with the tasks must be `static` or global – never a local variable of `main`.

ENSEA
Beyond Engineering

Demo: tick at 10 Hz

- Set `configTICK_RATE_HZ` to **10**: slices of 100 ms
- Both tasks never block: they alternate every slice
- The menu becomes sluggish, events arrive in bursts

What it shows

The limit of time-slicing. Next sessions: make tasks *wait* and let priorities decide – preemption.

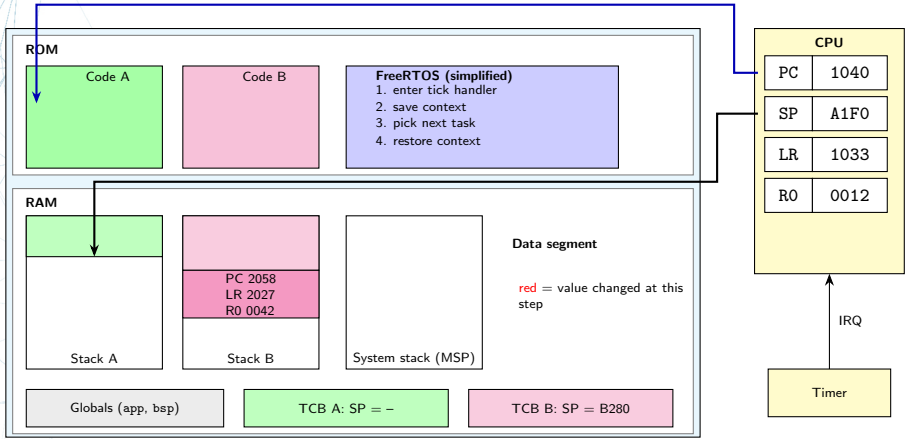
ENSEA
Beyond Engineering

One timer, two users: the HAL time base

- FreeRTOS uses `SysTick` for its tick
- The HAL (`HAL_GetTick`, `HAL_Delay`) also used `SysTick`
- In CubeMX: move the HAL **timebase** to a free timer (`TIM6`)
- Otherwise CubeMX warns at code generation

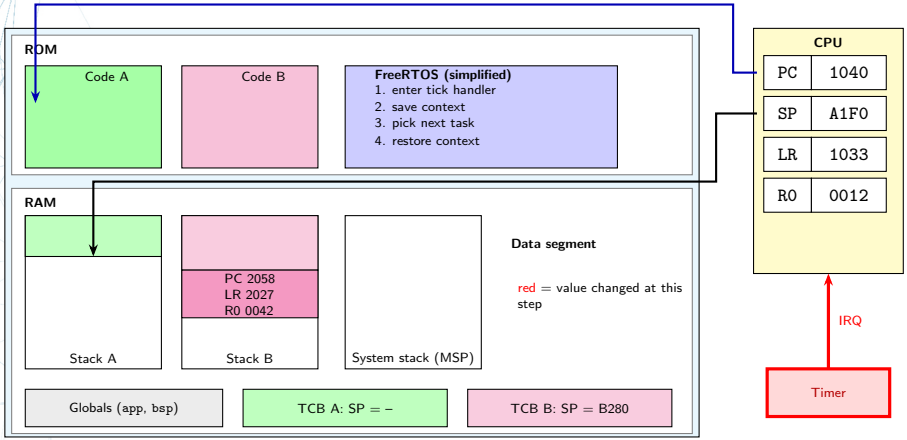
ENSEA
Beyond Engineering

Context switch: A → B → A



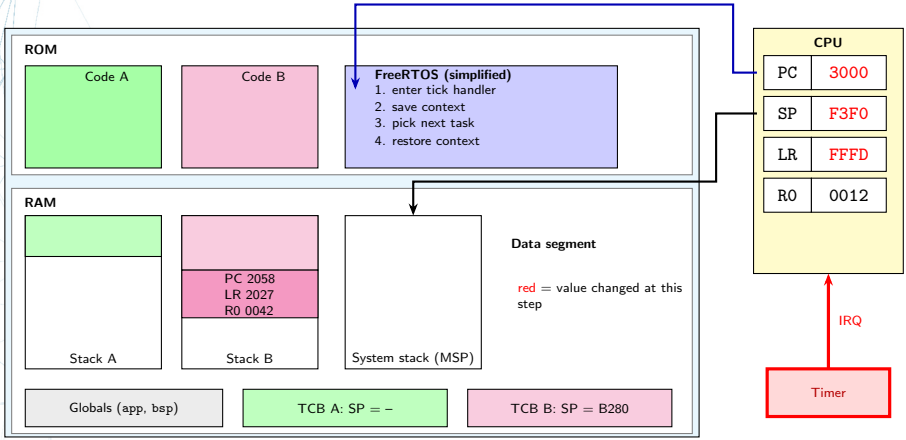
Task A is running: PC points into A's code, SP into A's stack.

Context switch: A → B → A



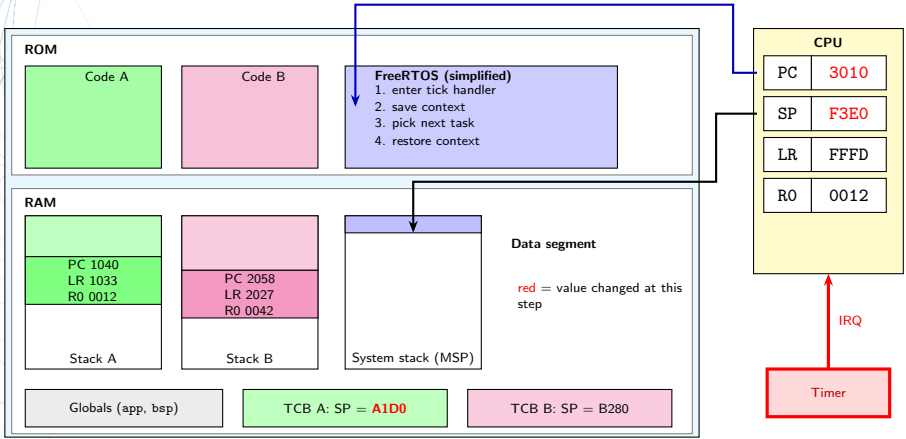
The timer (SysTick) raises an interrupt request.

Context switch: A → B → A



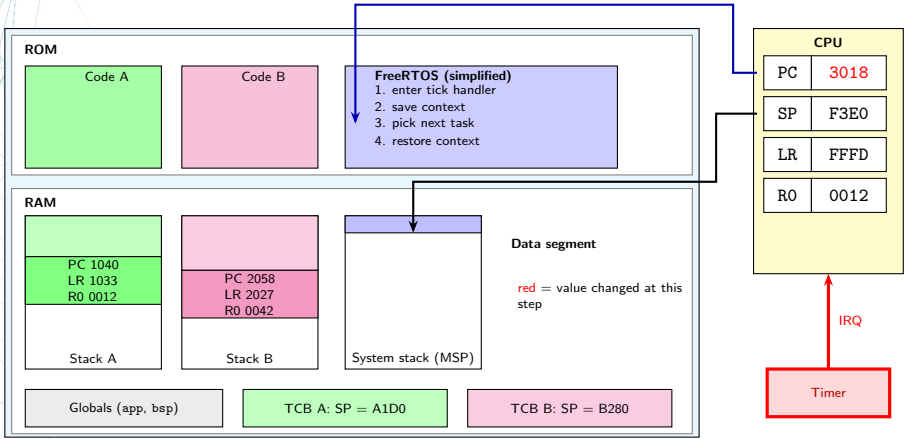
The CPU jumps to the tick handler in the FreeRTOS kernel, on the system stack (MSP).

Context switch: A → B → A



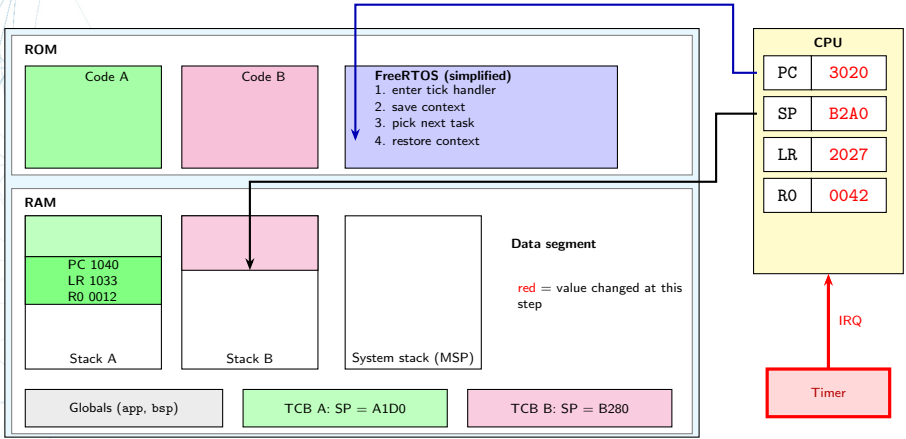
Save: A's registers are pushed on A's stack, and A's stack pointer is stored in its TCB.

Context switch: A → B → A



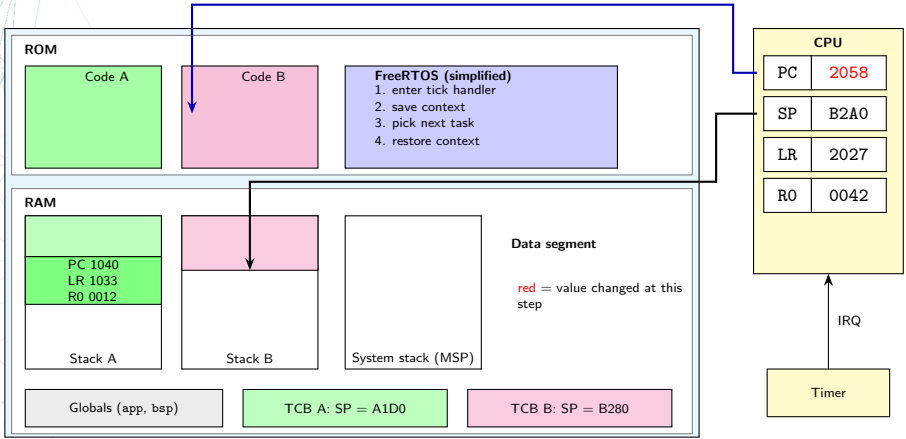
The scheduler picks the next task: B.

Context switch: A → B → A



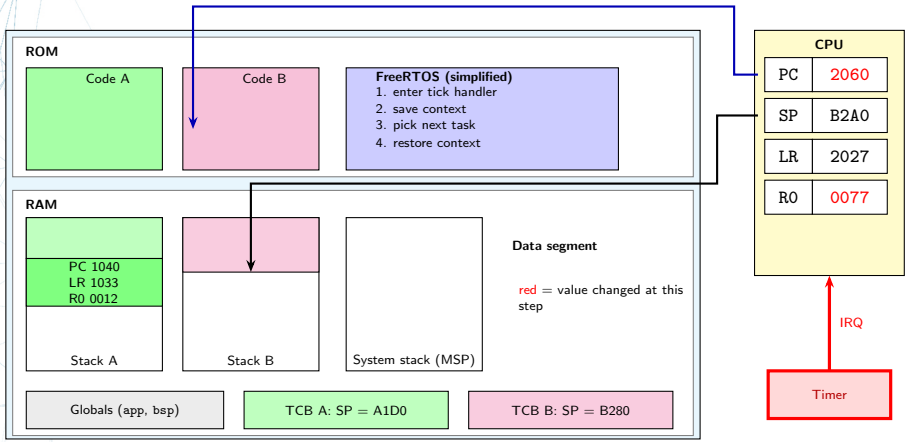
Restore: B's SP is loaded from its TCB, B's registers are popped from B's stack.

Context switch: A → B → A



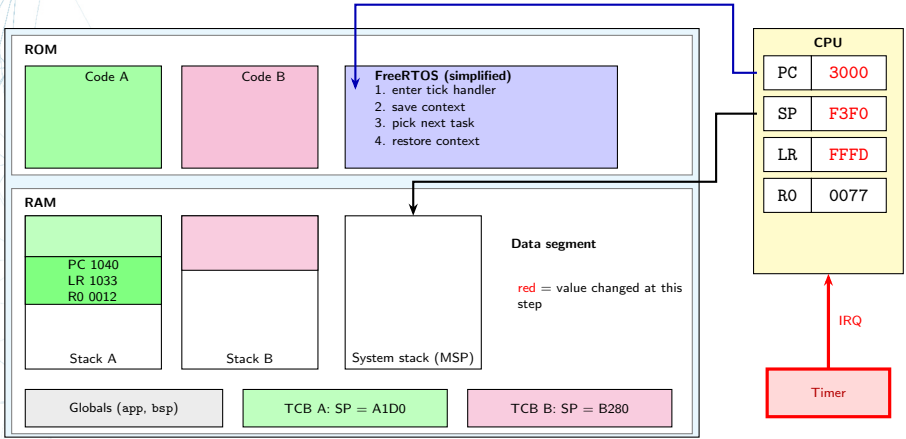
Return from the handler: PC points into B's code. B resumes exactly where it was stopped.

Context switch: A → B → A



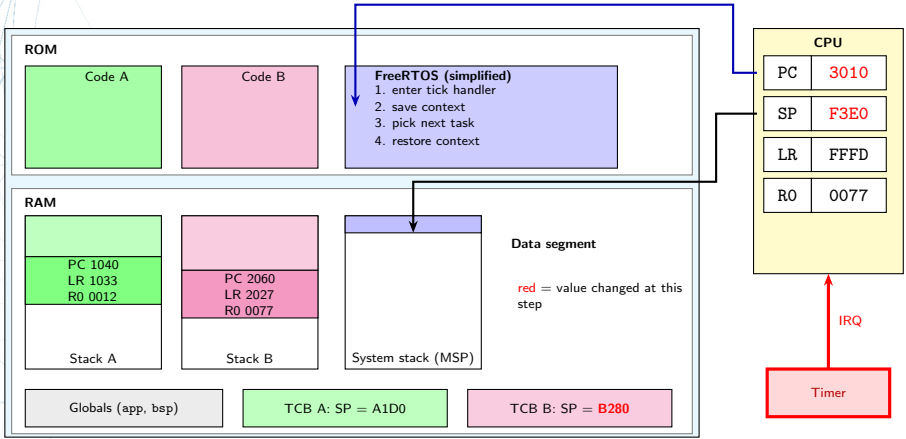
B runs for a while: its registers evolve (PC, R0). The timer fires again.

Context switch: A → B → A



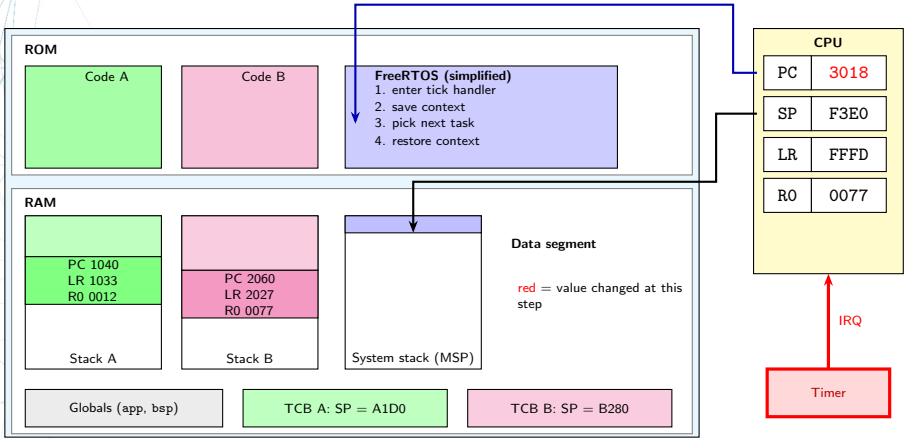
Same handler, same system stack.

Context switch: A → B → A



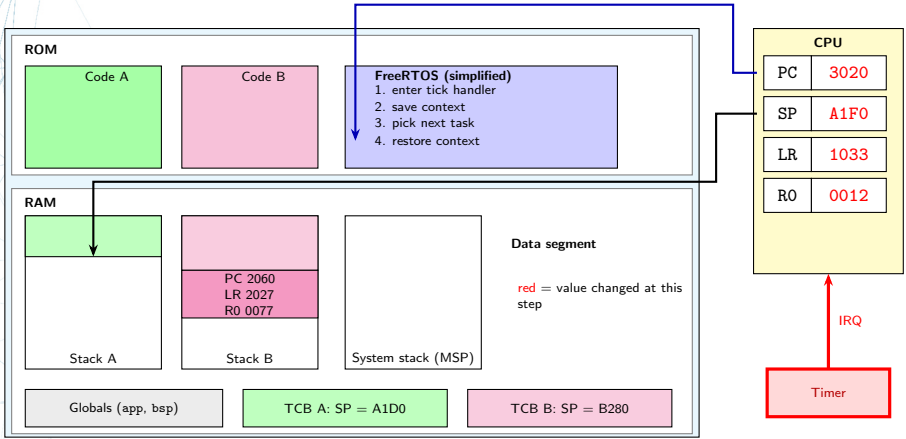
Save: B's *current* registers replace the old ones on B's stack; TCB B is updated.

Context switch: A → B → A



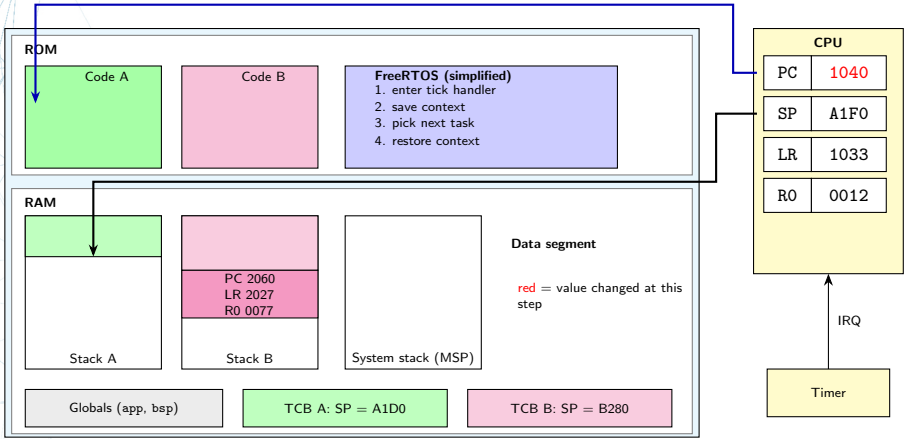
The scheduler picks the next task: A.

Context switch: A → B → A



Restore: A's registers are popped – the very same values as in the first step.

Context switch: A → B → A



Return: A resumes exactly where it was interrupted, with its own registers and its own stack.

Switching from one task to another (details)

- The tasks never notice: each one "believes" it owns the CPU
- The cost of a switch is not zero: saving, scheduling decision, restoring
  - One more reason not to make the tick too fast
- On Cortex-M: done in the `PendSV` exception, the lowest-priority interrupt

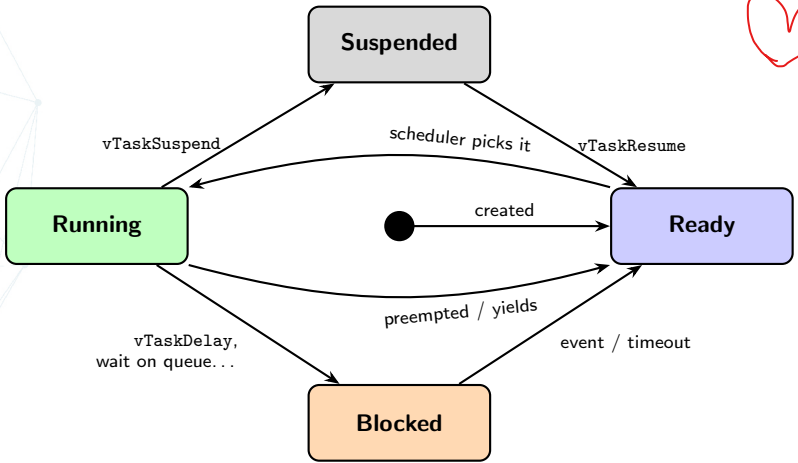
ENSEA
Beyond Engineering

A task is more than its code

- A task is: **code** + **context** (registers) + **entry point** + **state** + **priority**
- All of this is kept in a structure: the **TCB** (Task Control Block)
- Several tasks can share the same code, with different stacks
- In FreeRTOS, all tasks share the **same address space** (no memory protection)

ENSEA
Beyond Engineering

Task states



- Only **one** task is Running (single core). Blocked tasks do not use the CPU

Priorities

- From 0 to `configMAX_PRIORITIES - 1` – here 0 to 6
- **0 is the lowest** priority

En théorie jusqu'à 255 (8 bits)

Careful: opposite of the Cortex-M NVIC

- For FreeRTOS tasks, a bigger number is more urgent.
- For NVIC interrupts, **0 is the most urgent**.



- Preemptive rule: among Ready tasks, the highest priority runs.
- Same priority: time-sliced

ENSEA
Beyond Engineering

A task never returns

Infinite loop

```
void task_blink(void *unused) {
    for (;;) {
        /* ... */
    }
}
```

Finite loop, then delete

```
void task_init(void *unused) {
    for (int i = 0; i < 10; i++) {
        /* ... */
    }
    vTaskDelete(NULL);
}
```

- Either an infinite loop `for(;;)`, or finish with `vTaskDelete(NULL)`
- Returning from a task function is a bug: there is nowhere to return to

NULL veut dire ici que ça détruit la tâche en cours, sinon on peut mettre le registre de la fonction pour la supprimer (task B supprime task A)

The Idle task

- Created automatically by `vTaskStartScheduler()`, priority 0
- Runs when no other task is Ready
- In our demo, `task_bsp` and `task_app` **never block**: 100% CPU, Idle never runs
- A task that waits (Blocked state, `vTaskDelay`) *gives the CPU away* – Idle, or another task, uses it

ENSEA
Beyond Engineering

The BSP does not need 100 % of the CPU

```
void task_bsp(void *unused) {
    for (;;) {
        bsp_process(&bsp);                  /* poll buttons and encoder      */
        vTaskDelay(pdMS_TO_TICKS(5));       /* Blocked for 5 ms: CPU is free */
    }
}
```

- Buttons are polled: checking them every 5 ms is plenty (debounce works in milliseconds anyway)
- `task_bsp` now spends most of its time **Blocked**
- `task_app` still never blocks: it takes all the remaining CPU, so Idle still never runs

ENSEA
Beyond Engineering

Question: which task gets the higher priority?

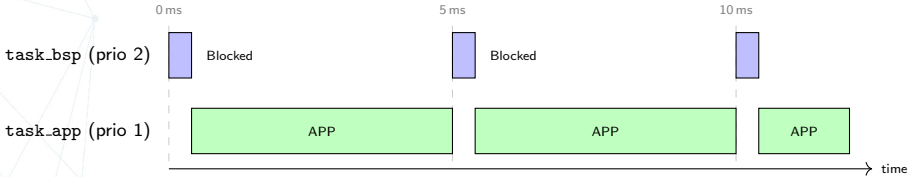
- `task_bsp`: short job every 5 ms, then Blocked
- `task_app`: never blocks (busy loop)

Question

With preemption, what happens if we give priority 2 to one task and 1 to the other?

ENSEA
Beyond Engineering

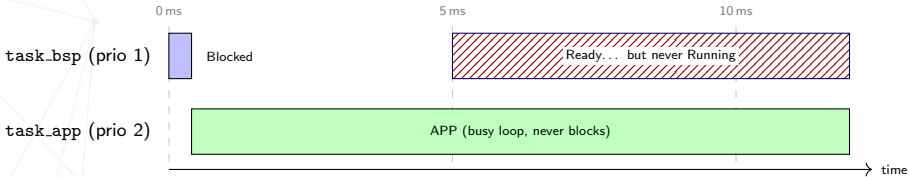
Case 1: BSP higher priority (BSP = 2, APP = 1)



- Every 5 ms the BSP wakes up and **preempts** the APP, does its short job, blocks again
- The APP gets everything else. Buttons respond within 5 ms, whatever the APP is doing
- **It works**

ENSEA
Beyond Engineering

Case 2: APP higher priority (APP = 2, BSP = 1)



- After its first delay, the BSP is Ready – but a higher-priority task is *always* Ready
- The BSP never runs again: **starvation**. No button, no encoder responds
- Same thing if the BSP had the higher priority *without* `vTaskDelay`: the APP would starve

ENSEA
Beyond Engineering

Rule of thumb for priorities

A high-priority task must block

The higher the priority, the **shorter** the job and the more often the task waits.

- A task that never blocks can only live at the **lowest** priority (above Idle)
- Typical: the task close to the hardware (BSP) is high priority and short; the application is lower
- Next sessions: the APP will block too (waiting for events in a queue) – and Idle will finally run

ENSEA
Beyond Engineering

CMSIS-OS vs. FreeRTOS

- CMSIS-OS (ARM) is an **interface**, not an implementation
  - Several implementations exist; FreeRTOS is one of them
- It is a thin layer of functions calling FreeRTOS functions
- It does not expose everything (e.g. run-time statistics)
- STM32CubeMX uses CMSIS-OS by default; raw FreeRTOS is still reachable

Our choice: FreeRTOS directly

- No right answer. For teaching: fewer indirections, and more control than CubeMX-generated code.
- Manual task creation = mandatory for Active Object paradigm

ENSEA
Beyond Engineering

Bypassing CMSIS in CubeMX

```
/* USER CODE BEGIN 2 */
xTaskCreate(task_bsp, "BSP", 256, NULL, 1, NULL);
xTaskCreate(task_app, "APP", 256, NULL, 1, NULL);
vTaskStartScheduler();          /* never returns */
/* USER CODE END 2 */

MX_FREERTOS_Init();             /* generated: never reached */
osKernelStart();                /* generated: never reached */
```

Démmare l'OS, boucle infinie

Forcément générer

- CubeMX generates `MX_FREERTOS_Init()` + `osKernelStart()` (and a `defaultTask` in `app_freertos.c`)
- We call `vTaskStartScheduler()` **before** them. It never returns, so the generated CMSIS code never runs

ENSEA
Beyond Engineering

Naming convention

- Function names are prefixed by their **return type**, then the module

  `x` `Task` `Create` `v` `Task` `Delay`

- `v`: `void` `x`: `BaseType_t` (or another type) `u`: unsigned `p`: pointer
- Then the module (`Task`, `Queue`, `Semaphore`...) and the action
- Constants follow the same idea: `pdPASS`, `pdMS_TO_TICKS`, `configXXX`

ENSEA
Beyond Engineering

xTaskCreate

```
BaseType_t xTaskCreate(TaskFunction_t pxTaskCode,   /* entry point            */
                       const char *pcName,          /* debug name             */
                       configSTACK_DEPTH_TYPE usStackDepth, /* in WORDS, not bytes */
                       void *pvParameters,          /* passed to the task     */
                       UBaseType_t uxPriority,      /* 0 = lowest             */
                       TaskHandle_t *pxCreatedTask);/* handle, or NULL        */
```

- Returns pdPASS, or an error if the heap is too small: **always check it**
- Stack depth is counted in 32-bit words: 256 words = 1 kB
- The handle is needed to act on a task later (delete, suspend, notify...)

ENSEA
Beyond Engineering

Delay, delete, suspend, resume

```
vTaskDelay(pdMS_TO_TICKS(100));      /* Blocked for 100 ms                        */
vTaskDelete(handle);                 /* remove a task (NULL = the calling task)   */
vTaskSuspend(handle);                /* Suspended until someone resumes it        */
vTaskResume(handle);                 /* back to Ready                             */
xTaskAbortDelay(handle);             /* wake a Blocked task before its timeout    */
```

- vTaskDelay is how a task politely gives the CPU away
- Next sessions: blocking on queues, notifications, semaphores and mutexes

ENSEA
Beyond Engineering

Summary

- An OS lets one CPU run several tasks, managing time, memory and synchronisation
- Real-time = predictable, not fast – and the RTOS alone does not guarantee it
- Cooperative → time-sliced → **preemptive**: the last one gives short, bounded latencies
- A task = code + context + state + priority, stored in a TCB; the tick triggers context switches
- A high-priority task must block, otherwise lower-priority tasks starve
- Our architecture paid off: only `main.c` changed
- Next: making tasks *wait*, and exchange data – queues

ENSEA
Beyond Engineering